

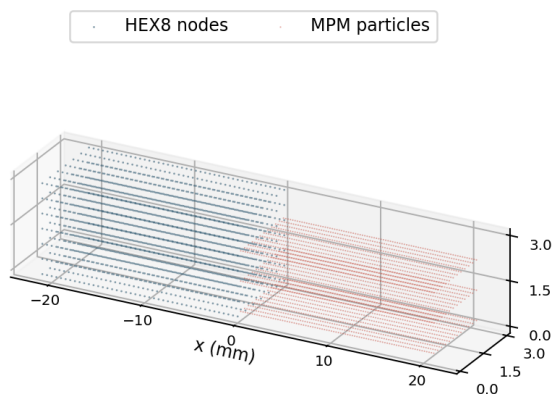
从碰撞直觉到 Taichi kernel

三维 CFEMP 算法与代码实现教程

HEX8 有限元 + 三维 MPM + 局部多网格接触

围绕仓库 ecjtusyy/CFEMP 的可复现实现

3D HEX8 FEM / MPM discretization



cross-section: 3 x 3 mm

左侧 HEX8 FEM，右侧三维 MPM 物质点；接触面位于 $x = 0$ 。

这份教程为谁写

面向已经学过高等数学、线性代数、数值 PDE，能读 Python/Taichi，并知道 FEM/MPM 基本名词，但希望把弱形式、P2G/G2P、接触投影和代码组织真正串起来的读者。阅读目标不是“看懂仓库”，而是最终能从空文件亲手重写一个可验证版本。

版本：2026-07-26 代码基线：main / merge commit 48ca466

阅读路线：先抓住三条主线

这个项目看起来同时包含连续体力学、有限元、物质点法、接触力学和 GPU/CPU kernel，但真正写代码时只需要抓住三条主线：状态放在哪里、一个时间步怎样推进、怎样证明推进没有写错。



CFEMP 的关键：把 FEM 接触表面也临时映射到 MPM 网格，再做局部动量投影

- 状态线：FEM 节点和 Gauss 点、MPM 粒子、临时背景网格分别保存什么。
- 时间线：P2G ☒ 接触投影 ☒ 应力与内力 ☒ 再投影 ☒ G2P。
- 验证线：单元几何、质量守恒、接触冲量、解析应力、分离时间、能量。

与你现有基础的衔接

你已经把 mpm88.py 理解为 P2G/G2P，也做过数值 PDE、误差与收敛实验。所以本文不会从 Python 语法讲起，而会重点补上：弱形式为什么产生内力公式、欧拉网格为什么每步清空、接触投影为什么守恒，以及这些公式怎样变成 Taichi field 和 kernel。

目录

阅读路线：先抓住三条主线	2
目录	3
1 算例到底在求什么	5
1.1 物理场景	5
1.2 你说的“像弹簧”哪里对，哪里要修正	5
1.3 三维计算为什么看起来像一维波	6
1.4 解析答案：代码首先要对得上的三组数	6
2 从控制方程到两个离散方法	7
2.1 共同的连续体骨架	7
2.2 FEM：节点随材料走，Gauss 点承担积分	7
2.3 MPM：粒子携带历史，网格只负责计算	7
3 CFEMP 的核心：局部多网格接触	8
3.1 不是惩罚弹簧，而是动量投影	8
3.2 一个接触网格点上的计算	8
3.3 为什么回写 FEM 时必须做质量归一化	8
4 一个时间步怎样推进	10
4.1 为什么接触要做两次	10
4.2 step() 是整个项目最值得先默写的函数	11
5 代码的类是怎样设计的	12
5.1 PlateImpact3DConfig：参数只放一处	12
5.2 ContactStep3D：单步返回值，不让 step() 偷藏状态	12
5.3 SimulationHistory3D：把验证数据与求解状态分开	12
5.4 TaichiCFEMPPlateImpact3D：为什么仍然是一个大类	13
6 初始化：先把几何和内存写对	14
6.1 HEX8 网格与节点编号	14
6.2 Gauss 积分为什么在 NumPy 里预计算	14
6.3 MPM 粒子为什么放在半格位置	14
6.4 Taichi field 不是 NumPy 数组的替代品	14
7 kernel 逐组拆解	15
7.1 形函数：先验证 partition of unity	15
7.2 P2G：为什么必须 atomic_add	15
7.3 应力更新：速度梯度是桥梁	15
7.4 内力：公式相同，积分载体不同	15
7.5 G2P：98% FLIP + 2% PIC	15
8 接触 kernel 逐行理解	16
8.1 _map_surface：把 FEM 表面变成临时的第二套网格状态	16

8.2 _project_contact: 四行就是 CFEMP 的灵魂	16
8.3 _scatter_contact: 为什么 FEM 使用负号	16
8.4 _gap: 几何接触与速度接触必须同时成立	16
9 Taichi 写法: 你真正需要掌握的部分	17
9.1 @ti.data_oriented 的意义	17
9.2 ti.static 和 ti.ndrange	17
9.3 为什么使用 f64	17
9.4 为什么原子加法顺序会导致末位轻微变化	17
9.5 时间步的量纲检查	17
10 benchmark、动画与测试怎样分工	18
10.1 benchmark_3d.py 不是求解器的一部分	18
10.2 animation 只读状态, 不参与求解	18
10.3 测试金字塔	19
11 当前结果怎样判断 “算对了”	20
11.1 哪些误差是正常离散误差	20
11.2 哪些现象一出现就说明代码错了	20
12 你怎样从空文件亲手写出来	21
12.1 第一遍: 只写接口, 不写公式	21
12.2 第二遍: 每写一个算子, 立即写一个局部测试	21
12.3 第三遍: 先小网格调通, 再换论文网格	21
13 最容易写错的十个地方	22
13.1 推荐的调试打印	22
14 读仓库时应该按什么顺序	23
14.1 关键代码定位	23
15 做完本教程后, 你应该能回答的问题	24
15.1 建议的亲手练习	24
15.2 下一版合理的工程架构	24
附录 A 最小伪代码	25
附录 B 运行命令	25
附录 C 参考资料	25

1 算例到底在求什么

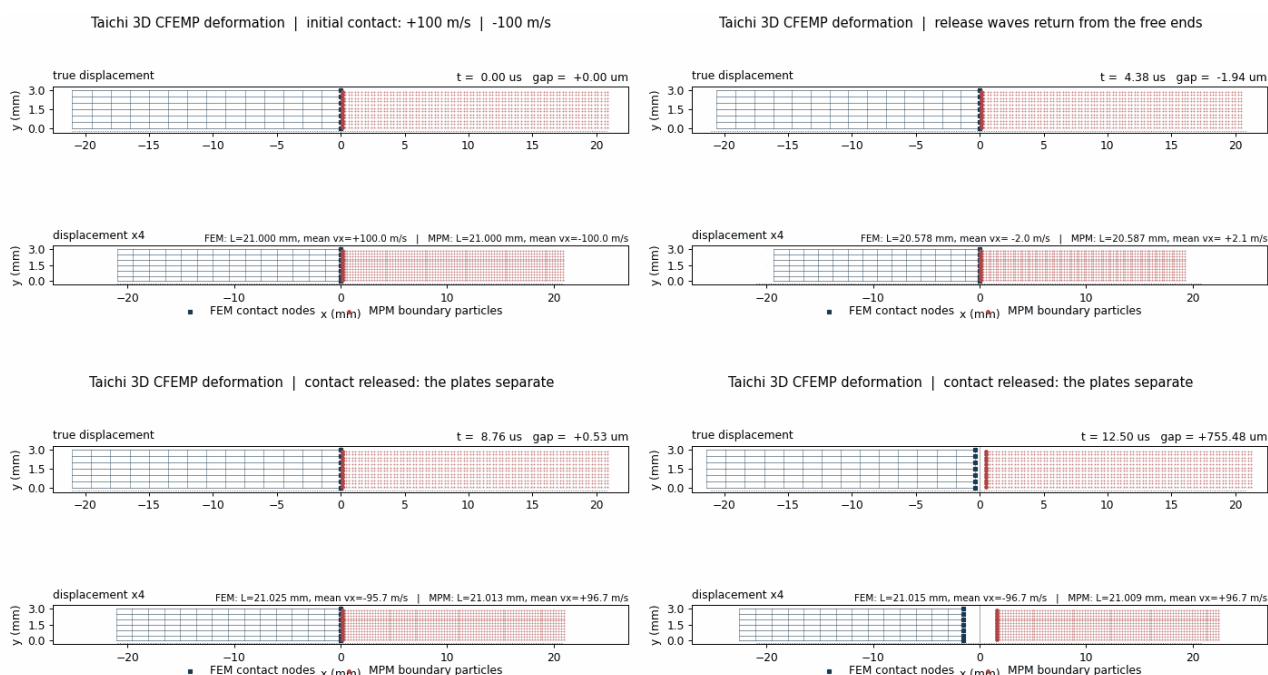
1.1 物理场景

两块尺寸完全相同的长方体板正碰。每块板长 21 mm，横截面为 3 mm × 3 mm。左板位于 [-21, 0] mm，用三维 HEX8 有限元离散；右板位于 [0, 21] mm，用三维 MPM 粒子离散。两者初始速度分别为 +100 m/s 和 -100 m/s，初始间隙为 0。

量	数值	代码位置
几何	21 × 3 × 3 mm	PlateImpact3DConfig
材料	E = 65 GPa, nu = 0, rho = 2750 kg/m ³	配置类
FEM	42 × 6 × 6 = 1512 个 HEX8	_hex8_mesh()
MPM	84 × 12 × 12 = 12096 个粒子	__init__()
背景网格	0.5 mm, 2254 个网格点	__init__()
时间	dt = 20 ns, 终止 15 us, 共 750 步	run()

1.2 你说的“像弹簧”哪里对，哪里要修正

把板想成弹簧是一个很好的第一直觉：碰撞把整体动能转成弹性应变能，随后板恢复长度并反弹。但它不是只有一个自由度的集中弹簧，而是由大量质量与弹性连接组成的分布式系统。接触面的节点先减速，后面的节点仍在向前运动，所以局部先压缩；这个压缩状态再以有限波速向自由端传播。

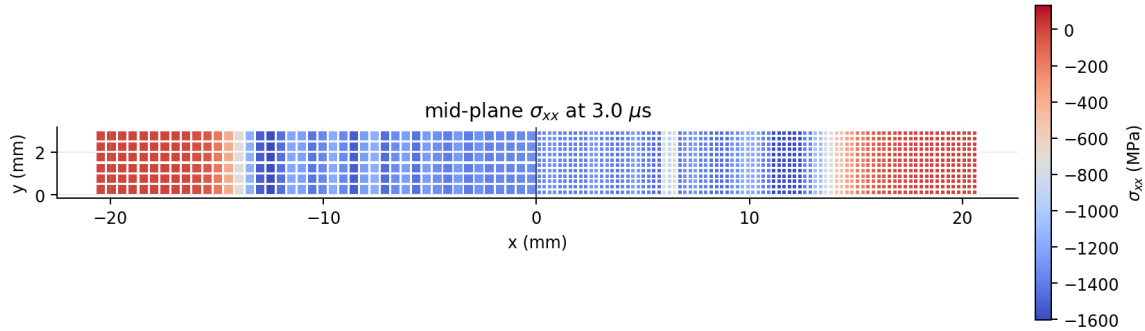


实际 Taichi 场生成的四个阶段。上半幅是真实位移，下半幅将位移放大 4 倍。

- 0 us：两板刚接触，左 +100 m/s，右 -100 m/s。
- 约 4.32 us：压缩波到达自由端，两板最短，整体平均速度接近 0。
- 4.32-8.64 us：自由端反射卸载波，板恢复长度并开始反向运动。
- 约 8.68 us：界面分离；之后左板向左、右板向右。

1.3 三维计算为什么看起来像一维波

几何、速度、应力和离散全部是三维的，但 y、z 侧面施加平面应变约束，而且载荷在横截面上完全均匀。因此解沿横截面保持均匀，主要只随 x 变化。这正适合作为三维代码基准：代码必须正确处理三维状态，而结果又能与一维解析纵波对照。



3 us 时中截面的轴向应力。横截面均匀，压缩区从界面向两端扩展。

1.4 解析答案：代码首先要对得上的三组数

$$c = \sqrt{\frac{E(1-\nu)}{\rho(1+\nu)(1-2\nu)}} = 4861.72 \text{ m/s}$$

$$\sigma_c = -\rho c v_0 = -1336.97 \text{ MPa}$$

$$t_s = \frac{2L}{c} = 8.6389 \mu\text{s}$$

受约束纵波速度、接触压应力和分离时间。

这三式非常重要，因为它们来自独立解析力学，而不是代码内部自治。如果程序的应力、波前位置和分离时刻都能逼近这些值，说明接触与应力波传播至少在这个基准上是可信的。

物理范围

当前是论文的理想线弹性基准。1.34 GPa 对许多真实铝合金已超过屈服，所以它能验证算法，却不能替代真实材料实验。要做真实碰撞，还需 J2 塑性、硬化、温升或损伤模型。

2 从控制方程到两个离散方法

2.1 共同的连续体骨架

FEM 和 MPM 解决的是同一套动量守恒与本构关系。区别不在物理方程，而在“状态存在哪里、积分在哪里做”。无体力时，强形式可以写成质量密度乘加速度等于应力散度；线弹性材料由小应变和 Hooke 定律闭合。

$$\rho \dot{\mathbf{v}} = \nabla \cdot \boldsymbol{\sigma}$$

$$\dot{\boldsymbol{\epsilon}} = \text{sym}(\nabla \mathbf{v})$$

$$\boldsymbol{\sigma} = \lambda \text{tr}(\boldsymbol{\epsilon}) \mathbf{I} + 2\mu \boldsymbol{\epsilon}$$

动量方程、速度梯度到应变率、各向同性线弹性本构。

2.2 FEM：节点随材料走，Gauss 点承担积分

FEM 是拉格朗日描述：网格节点与材料一起运动。先用形函数插值速度，再把动量方程乘测试函数并分部积分，得到节点质量和内力。代码使用 lumped mass，每个 HEX8 单元的质量平均分给八个节点。

$$\mathbf{v}(\mathbf{x}) \approx \sum_a N_a(\mathbf{x}) \mathbf{v}_a$$

$$m_a \approx \sum_{e \ni a} \frac{\rho V_e}{8}$$

$$\mathbf{f}_a^{\text{int}} = - \sum_e \sum_q |J_{eq}| \boldsymbol{\sigma}_{eq} \nabla N_a$$

FEM 的插值、集中质量和节点内力。

项目使用 $2 \times 2 \times 2$ Gauss 全积分。原论文程序使用单点积分加沙漏控制；因为论文没有给全沙漏参数，这里选择全积分，牺牲少量速度换取可复现和稳定。

2.3 MPM：粒子携带历史，网格只负责计算

MPM 的粒子也是拉格朗日点，携带质量、体积、速度、应变和应力；规则背景网格是临时欧拉计算器。每一步先 P2G，把粒子质量与动量投到网格；在网格上计算速度、内力与接触；最后 G2P，把变化传回粒子，然后清空网格。

$$m_I = \sum_p N_{Ip} m_p, \quad \mathbf{p}_I = \sum_p N_{Ip} m_p \mathbf{v}_p$$

$$\mathbf{f}_I^{\text{int}} = - \sum_p V_p \boldsymbol{\sigma}_p \nabla N_{Ip}$$

$$\mathbf{v}_p^{n+1} = \alpha \mathbf{v}_p^{\text{FLIP}} + (1 - \alpha) \mathbf{v}_p^{\text{PIC}}$$

P2G、网格内力和 FLIP/PIC 混合 G2P；本实现 $\alpha = 0.98$ 。

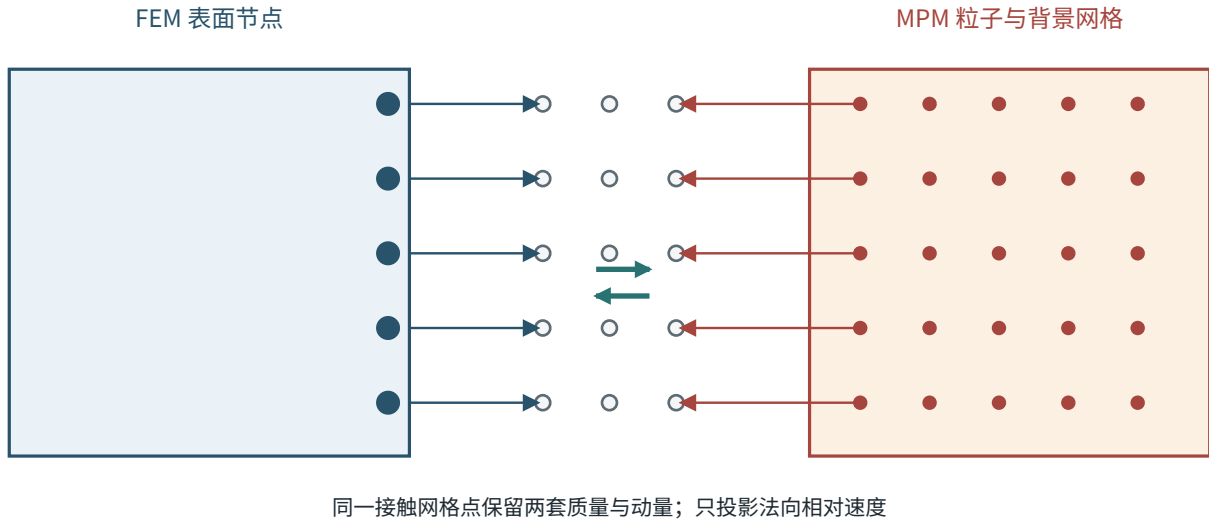
为什么网格每步清空

背景网格不保存材料历史，因此不会随着大变形被拉坏。质量、应力等历史跟着粒子走；网格只在当前步提供一个规整的微分与接触计算场。

3 CFEMP 的核心：局部多网格接触

3.1 不是惩罚弹簧，而是动量投影

惩罚法在界面间隙为负时施加 $k \cdot \text{penetration}$ 型接触力，刚度 k 需要调参。本项目的 CFEMP 主路径不使用惩罚刚度，而是在共享背景网格点上保留 FEM 和 MPM 两套质量、动量，然后把不满足不可穿透条件的相对法向速度直接投影到可行集合。



3.2 一个接触网格点上的计算

设映射到网格点 l 的 FEM 质量和速度为 m_f 、 $\mathbf{v}_f$ ，MPM 质量和速度为 m_m 、 $\mathbf{v}_m$ ，法向 $\mathbf{n}$ 从 FEM 指向 MPM。只有当两物体在法向上继续靠近时才修正。

$$u_n = (\mathbf{v}_f - \mathbf{v}_m) \cdot \mathbf{n}$$

$$m_r = \frac{m_f m_m}{m_f + m_m}, \quad \mathbf{J} = m_r u_n \mathbf{n}$$

$$\mathbf{v}'_f = \mathbf{v}_f - \frac{\mathbf{J}}{m_f}, \quad \mathbf{v}'_m = \mathbf{v}_m + \frac{\mathbf{J}}{m_m}$$

$$(\mathbf{v}'_f - \mathbf{v}'_m) \cdot \mathbf{n} = 0$$

质量加权的法向动量投影。

MPM 网格得到 $+\mathbf{J}$ ，FEM 得到 $-\mathbf{J}$ ，所以界面冲量严格等大反向。投影后相对法向速度恰好为 0；切向速度没有修改，因此本算例是无摩擦滑动接触。

3.3 为什么回写 FEM 时必须做质量归一化

一个 FEM 表面节点会通过三线性函数映射到最多八个接触网格点。网格冲量回到 FEM 节点时，如果只乘一次形函数，会因为多个节点共同贡献质量而重复或漏算。代码使用“节点映射质量 / 网格上的 FEM 总质量”作为归一化权重，从而保证所有 FEM 节点冲量之和等于 $-\mathbf{J}$ 。

$$\Delta \mathbf{p}_a = - \sum_I \frac{m_a N_{Ia}}{m_{f,I}} \mathbf{J}_I$$

$$\sum_a \Delta \mathbf{p}_a = - \sum_I \mathbf{J}_I$$

归一化回映射与界面冲量守恒。

你可以怎样理解这个投影

它等价于：在总动量不变的前提下，寻找距离原速度最近、又满足“不再相互靠近”的速度。这里的“最近”不是普通欧氏距离，而是按质量加权的动能范数。质量大的物体速度改得少，质量小的改得多。

4 一个时间步怎样推进

读数值代码最容易迷失在几十个 kernel 之间。正确方法是先只看 `step()`，把它当成算法目录；等调用顺序清楚后，再进入每个 kernel。



4.1 为什么接触要做两次

第一次投影发生在 P2G 后，修正当前速度，随后用这组速度更新应变和应力；第二次投影发生在内力自由预测后，修正 trial velocity。如果只做第一次，内力预测可能重新制造穿透速度；如果只做第二次，应力更新可能已经使用了错误的接触速度。

4.2 step() 是整个项目最值得先默写的函数

缩写后的 step() 主线

```
def step(self):
    close = self._gap() <= self.config.contact_tolerance

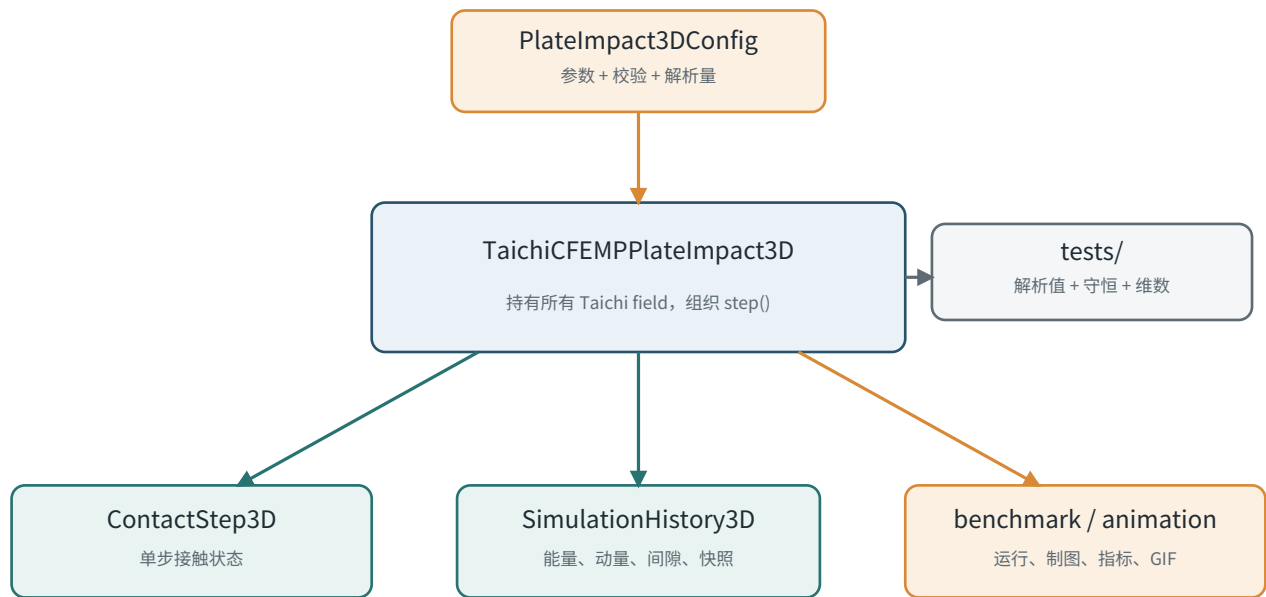
    self._reset_grid()
    self._p2g()
    self._compute_grid_velocity(copy_before=1)
    self._contact_stage(use_trial=0, enabled=close)

    self._update_fem_stress()
    self._update_mpm_stress()
    self._clear_forces()
    self._fem_internal_force()
    self._mpm_internal_force()
    self._make_trial_state()
    self._contact_stage(use_trial=1, enabled=close)

    self._finish_step()
    self._g2p()
    self.time += self.dt
```

你自己重写时，先把这些函数留成空壳并打印调用顺序。只有主线稳定后，才逐个实现 P2G、内力和接触。不要一开始就在一个 kernel 里塞完所有功能。

5 代码的类是怎样设计的



5.1 PlateImpact3DConfig: 参数只放一处

配置类是 frozen dataclass，负责几何、材料、离散、时间步和派生物理量。validate() 把不合法状态挡在求解器构造之前，例如时间步、泊松比、网格整除性和匹配网格条件。

配置类的最小骨架

```

@dataclass(frozen=True)
class PlateImpact3DConfig:
    length: float = 21e-3
    young: float = 65e9
    density: float = 2750.0
    fem_element_size: float = 0.5e-3
    particle_spacing: float = 0.25e-3
    dt: float = 2e-8

    @property
    def wave_speed(self):
        return sqrt(self.constrained_modulus / self.density)

    def validate(self):
        ...
  
```

5.2 ContactStep3D: 单步返回值，不让 step() 偷藏状态

它只记录当前步激活接触点数、几何间隙、两阶段冲量和守恒残差。这样 run() 不需要理解接触 kernel 的内部细节，也能判断是否仍处于接触并记录分离时间。

5.3 SimulationHistory3D: 把验证数据与求解状态分开

历史对象保存每一步的能量、动量、间隙和接触信息，并在指定时刻保存快照。求解器只负责推进；benchmark 负责读取 history、画图和计算误差。这比在 kernel 里直接写 CSV 更容易测试，也更容易做参数扫描。

5.4 TaichiCFEMPPlateImpact3D：为什么仍然是一个大类

Taichi field 的 shape 在编译 kernel 前就要确定，因此 `__init__` 先构造网格和粒子，再创建全部 field。当前算例只有两个物体和一个界面，大类可以保持调用顺序直观。如果扩展到多物体、摩擦和曲面接触，应拆成 `FEMBody3D`、`MPMBody3D`、`ContactCoupler3D` 和 `Diagnostics`。

设计原则

配置是不可变输入；Taichi field 是可变求解状态；`ContactStep` 是单步事件；`History` 是时间序列；`benchmark/animation` 是外部观察者。把这五种角色混在一个类里，后面一定会难以测试。

6 初始化：先把几何和内存写对

6.1 HEX8 网格与节点编号

FEM 网格按 i 、 j 、 k 三重循环生成。节点索引 $\text{node}(i,j,k)$ 把三维整数坐标压成一维下标；每个单元按固定右手序连接八个角点。连接顺序一旦错误，Jacobian 可能为负，内力方向也会错。

节点编号与一个 HEX8 连接

```
def node(i, j, k):
    return (i * (ny + 1) + j) * (nz + 1) + k

conn = [
    node(i, j, k),
    node(i+1, j, k),
    node(i+1, j+1, k),
    node(i, j+1, k),
    node(i, j, k+1),
    node(i+1, j, k+1),
    node(i+1, j+1, k+1),
    node(i, j+1, k+1),
]
```

6.2 Gauss 积分为什么在 NumPy 里预计算

参考构型中的节点坐标和 HEX8 梯度在本算例中不随时间重算。因此构造阶段用 NumPy 计算每个单元、每个 Gauss 点的全局梯度和 $\det(J)$ ，再一次性拷入 Taichi field。这样 kernel 里只做真正的时间推进。

- 检查 1：所有 $\det(J) > 0$ 。
- 检查 2：所有 Gauss 点 $\det(J)$ 之和等于总体积。
- 检查 3：常量速度场的梯度必须为 0。
- 检查 4：仿射速度场的梯度应被精确恢复。

6.3 MPM 粒子为什么放在半格位置

粒子坐标使用 $(i+0.5) \cdot dp$ ，因此粒子位于小体素中心。粒子体积为 dp^3 ，质量为 $\rho \cdot dp^3$ 。最左粒子中心在 $dp/2$ ，计算物理间隙时要减去半个粒子间距，否则明明接触的两块板会被误判为有缝。

6.4 Taichi field 不是 NumPy 数组的替代品

状态	shape	物理含义
fem_x, fem_v	(2107, 3)	FEM 节点位置与速度
fem_stress	(1512, 8, 6)	单元 $\times$ Gauss 点 $\times$ 六应力分量
mpm_x, mpm_v	(12096, 3)	粒子位置与速度
mpm_stress	(12096, 6)	粒子应力
grid_mass	(46, 7, 7)	临时背景网格质量
contact_fem_mass	(46, 7, 7)	FEM 表面映射质量

求解循环内部只访问 Taichi field；`to_numpy()` 只在快照、指标和动画阶段使用。如果每个时间步都在 Python 和 Taichi 之间来回拷贝，性能会迅速崩掉。

7 kernel 逐组拆解

7.1 形函数：先验证 partition of unity

MPM 使用规则六面体网格的三线性形函数。每个粒子只影响所在 cell 的 $2 \times 2 \times 2$ 个节点。写完 `_mpm_shape()` 后，第一件事不是跑碰撞，而是随机取局部坐标检查八个权重之和等于 1、八个梯度之和等于 0。

7.2 P2G：为什么必须 `atomic_add`

`_p2g()` 的核心

```
for p in range(n_particles):
    base = cell_of(mpm_x[p])
    for offset in ti.static(ti.grouped(ti.ndrange(2, 2, 2))):
        w, _ = shape(mpm_x[p], base, offset)
        I = base + offset
        ti.atomic_add(grid_mass[I], w * mpm_mass[p])
        for d in ti.static(range(3)):
            ti.atomic_add(
                grid_momentum[I][d],
                w * mpm_mass[p] * mpm_v[p][d],
            )
```

多个粒子会同时写同一网格点，所以并行 kernel 中必须原子累加。最小自检是 P2G 前后总质量和总动量相等。若这一步不守恒，后面的接触和能量全部没有意义。

7.3 应力更新：速度梯度是桥梁

FEM 用节点速度和预计算的 `grad(N)` 构造速度梯度；MPM 用背景网格速度和网格形函数梯度构造。两边随后走同一个 `_stress_from_strain()`，保证材料模型一致。

7.4 内力：公式相同，积分载体不同

FEM 在八个 Gauss 点积分，MPM 在每个粒子处积分。两者的内力形式都是 $-\text{volume} \times \text{stress tensor} \times \text{shape gradient}$ 。自检时对所有节点或网格点求和，内部力合力应接近 0。

7.5 G2P：98% FLIP + 2% PIC

纯 PIC 稳定但耗散大；纯 FLIP 保存动量变化但容易保留网格噪声。本实现以 0.98 的比例使用 FLIP，以 0.02 的 PIC 轻微抑制振荡。粒子位置使用 PIC 网格速度推进，避免用带历史噪声的 FLIP 速度直接搬运位置。

8 接触 kernel 逐行理解

8.1 _map_surface: 把 FEM 表面变成临时的第二套网格状态

49 个 FEM 接触面节点通过与 MPM 相同的三线性形函数映射到背景网格，累加 `contact_fem_mass`、`contact_fem_momentum` 和法向。这一步没有改变任何速度，只是把 FEM 表面“翻译”为背景网格语言。

8.2 _project_contact: 四行就是 CFEMP 的灵魂

接触点上的质量投影

```
fem_velocity = fem_momentum / fem_mass
mpm_velocity = grid_momentum / mpm_mass
closing = dot(fem_velocity - mpm_velocity, normal)

if closing > 0.0:
    reduced_mass = fem_mass * mpm_mass / (fem_mass + mpm_mass)
    impulse = reduced_mass * closing * normal
    grid_momentum += impulse
```

符号最容易写反。当前法向从 FEM 指向 MPM，即 $+x$ 。左板向右、右板向左时， $(v_f - v_m) \cdot n = 200 > 0$ ，说明正在闭合；MPM 加 $+J$ 后向右减速，FEM 回写 $-J$ 后向左减速。

8.3 _scatter_contact: 为什么 FEM 使用负号

网格端已经加了 $+J$ ，所以 FEM 必须得到 $-J$ 。归一化权重保证共享同一网格点的多个 FEM 表面节点只分配一次总冲量。代码把 `grid_total + fem_total` 的范数保存为 `balance_residual`，测试要求小于 $1e-15 \text{ kg} \cdot \text{m/s}$ 。

8.4 _gap: 几何接触与速度接触必须同时成立

速度投影只在 `gap` 小于容差时启用；而在启用后，只有 `closing > 0` 的点才产生冲量。这避免相距很远的物体因为投影支撑域重叠而被错误耦合，也避免已经分离的物体被粘住。

接触调试顺序

先只用两个质量点验证投影公式；再用一个 FEM 节点映射到一个网格点；然后验证多节点归一化；最后才接入完整 P2G 和三维表面。直接在完整板碰撞中调符号，几乎一定会迷失。

9 Taichi 写法：你真正需要掌握的部分

9.1 @ti.data_oriented 的意义

它允许类方法 kernel 直接访问 self.fem_x 等 Taichi field。Python 负责构造、调度和少量诊断；大规模循环由 Taichi 编译。

9.2 ti.static 和 ti.ndrange

$2 \times 2 \times 2$ 的邻域大小在编译期固定，因此用 ti.static 展开八次循环。大粒子循环仍是运行时并行循环。正确区分这两者，代码既清晰又容易编译优化。

9.3 为什么使用 f64

接触冲量守恒和解析误差验证需要较高精度，因此 CPU 后端使用 f64。如果改用 GPU，很多消费级设备的 f64 性能很弱；这时应重新设置误差阈值，而不是假装结果完全相同。

9.4 为什么原子加法顺序会导致末位轻微变化

并行 reduction 的累加顺序不固定，浮点加法又不满足结合律。所以总动量误差可能在 $3.7e-13$ 附近略微变化。测试使用数量级阈值，而不是把某个末位小数写死。

9.5 时间步的量纲检查

$$\Delta t \leq 0.5 \frac{h}{c}$$

$$h = 0.5 \text{ mm}, \quad c = 4861.72 \text{ m/s}, \quad 0.5h/c = 51.42 \text{ ns}$$

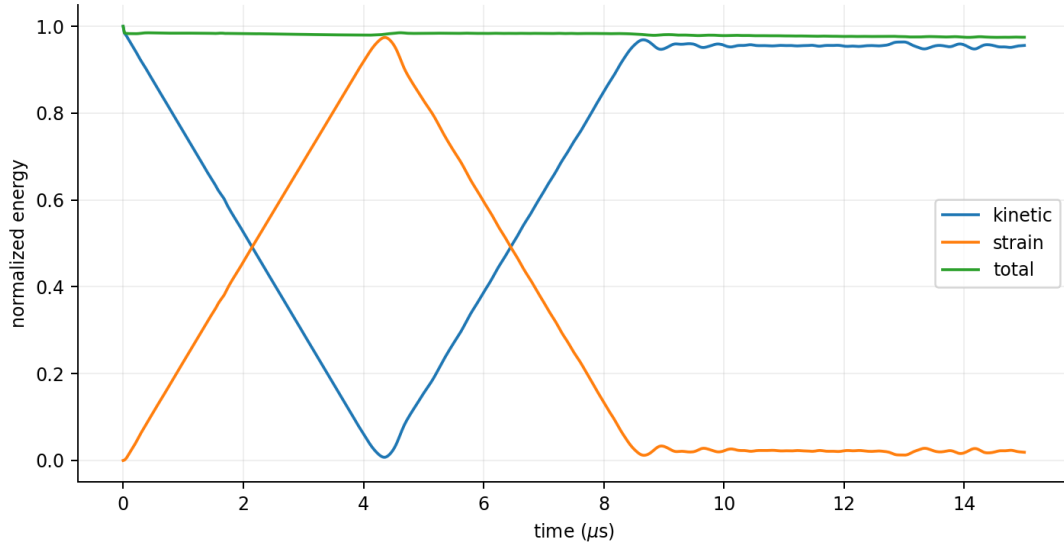
$$\Delta t = 20 \text{ ns} \Rightarrow \text{CFL} \approx 0.194$$

当前时间步小于显式稳定性上限。

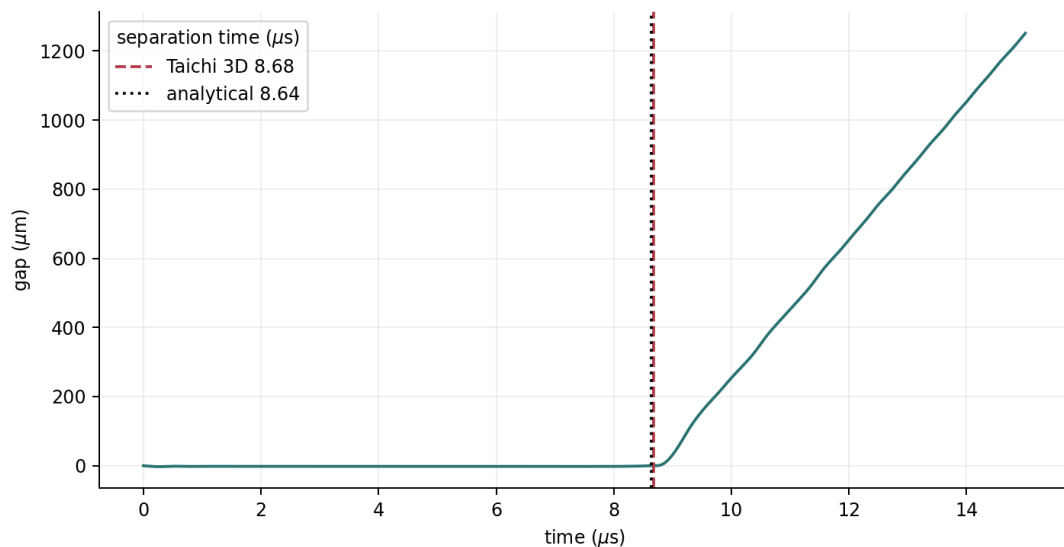
10 benchmark、动画与测试怎样分工

10.1 benchmark_3d.py 不是求解器的一部分

它实例化 solver、运行 750 步、读取 history、生成图片和 metrics.json。解析应力与分离时间在求解器模块中作为独立函数提供，因此测试和 benchmark 共用同一物理定义。



动能与应变能相互转换；总能量最大误差约 2.55%。



数值分离时间 8.68 us，与解析 8.6389 us 接近。

10.2 animation 只读状态，不参与求解

动画脚本在若干目标时刻把 Taichi field 转为 NumPy，取三维模型的中截面。上半幅显示真实位移；下半幅按 $x_{\text{vis}} = x_0 + 4(x - x_0)$ 放大。它不会把放大后的坐标写回求解器，因此不会污染数值结果。

10.3 测试金字塔

层级	测试问题	失败说明什么
几何单元	Jacobian、体积、HEX8 形状	网格编号或梯度错误
局部算子	P2G 守恒、仿射 patch、法向投影	kernel 公式或符号错误
界面守恒	FEM/MPM 冲量等大反向	回写归一化错误
基准解	应力、分离时间、能量	算法整体推进错误
物理验证	真实实验	当前项目尚未完成

CI 通过只能证明代码在两个 Python 版本上满足这些检查，不能自动证明真实材料模型正确。Verification 与 Validation 必须分开。

11 当前结果怎样判断 “算对了”

指标	解析/目标	本实现	判断
接触压应力	-1336.97 MPa	-1359.23 MPa	误差 1.66%
分离时间	8.6389 us	8.6800 us	误差 0.48%
最大 FEM 压缩	约 0.4319 mm	约 0.422 mm	误差约 2.3%
总能量	常数	最大误差 2.55%	显式计算可接受
总动量	0	归一化误差 $3.7\text{e-}13$	接近舍入误差
界面冲量	等大反向	残差 $8.7\text{e-}19$	守恒
最大穿透	0	2.806 um	约 0.56% 网格尺寸

11.1 哪些误差是正常离散误差

- 接触面处 MPM 与 FEM 离散方式不对称，会出现局部应力振荡。
- 显式时间积分、FLIP/PIC 混合和有限网格共同造成能量误差。
- 接触检测使用有限容差，因此存在数微米级的数值穿透。
- 并行原子累加会改变最后几个浮点末位。

11.2 哪些现象一出现就说明代码错了

- 两板接触后继续以原速度相互穿过：接触符号或 enabled 判据错误。
- FEM 和 MPM 同时得到同方向冲量：回写负号错误。
- 总质量随时间变化：P2G、粒子体积或边界越界错误。
- 无外力时总动量明显漂移：atomic、冲量归一化或边界条件错误。
- 横向速度不再接近 0：平面应变边界漏施加。
- 减小 dt 后误差不降：空间离散、接触算法或实现逻辑有问题。

12 你怎样从空文件亲手写出来

最有效的路线不是复制当前 1000 行，而是按下面八个里程碑逐层增加复杂度。每个阶段只有在自检通过后才能进入下一阶段。

阶段	只实现什么	必须通过的自检
0. 两质量点	一维质量加权接触投影	投影后相对速度为 0；总动量不变
1. 规则网格形函数	3D trilinear N 与 grad(N)	sum(N)=1；sum(grad N)=0
2. 纯 MPM	P2G、网格速度、G2P	匀速平移不变；质量和动量守恒
3. 单个 HEX8	Gauss 梯度、质量、线弹性	Jacobian 正；patch test；内力自平衡
4. FEM 板	多单元显式推进	无载匀速平移；应力波速度正确
5. 单网格点耦合	FEM 表面映射与归一化回写	网格 + FEM 冲量残差接近 0
6. 完整 CFEMP	双阶段接触 + MPM/FEM 内力	应力和分离时间接近解析值
7. 诊断与动画	history、metrics、GIF、CI	重复运行可复现；23 项测试通过

12.1 第一遍：只写接口，不写公式

建议的空壳

```
class Solver:
    def __init__(self, config): ...
    def reset_grid(self): ...
    def p2g(self): ...
    def contact(self, use_trial): ...
    def update_stress(self): ...
    def assemble_force(self): ...
    def predict(self): ...
    def g2p(self): ...
    def step(self): ...
    def run(self): ...
```

先让 step() 能按顺序调用这些空函数，并用一个极小配置运行 10 步。这样类关系、数据流和测试入口先稳定下来。

12.2 第二遍：每写一个算子，立即写一个局部测试

不要等完整板碰撞跑起来后再统一测试。shape、P2G、HEX8、contact projection 都有独立的精确性质，局部测试失败时定位范围只有十几行；完整算例失败时定位范围是整个求解器。

12.3 第三遍：先小网格调通，再换论文网格

调试配置可先用 $4 \times 1 \times 1$ 个 FEM 单元和几十个粒子。确认质量、速度、应力和冲量后，再换成 1512 单元与 12096 粒子。低配电脑上尤其要避免每次修改都跑完整 750 步。

13 最容易写错的十个地方

序号	错误	最快自检
1	法向方向与 closing 符号不一致	用初始 +100/-100 代入手算，必须得到 closing=200
2	FEM 冲量回写忘记负号	每步检查 grid_impulse + fem_impulse
3	回写不除 contact_fem_mass	多表面节点时冲量会放大
4	P2G 未清空网格	质量会每步累积
5	并行写网格不用 atomic_add	结果随机且不守恒
6	MPM 物理边界等同于粒子中心	间隙多出 dp/2
7	HEX8 节点顺序错	det(J) 负或内力方向异常
8	工程剪应变与张量剪应变混用	剪切应力会差 2 倍
9	用 FLIP 速度直接推进位置	粒子位置容易积累噪声
10	把测试通过当作真实实验验证	区分 Verification 与 Validation

13.1 推荐的调试打印

每个阶段只保留少量守恒量

```
print("mass", fem_mass + mpm_mass)
print("momentum", total_momentum)
print("energy", kinetic + strain)
print("gap", interface_gap)
print("active", contact_nodes)
print("impulse residual", norm(J_grid + J_fem))
```

不要一开始打印所有粒子。先看全局量是否守恒，再定位到接触网格点，最后才检查单个粒子或单元。

14 读仓库时应该按什么顺序

顺序	文件 / 函数	阅读目标
1	README.md	明确问题、参数、结果和范围
2	PlateImpact3DConfig	认识全部尺度和稳定性条件
3	TaichiCFEMPPlateImpact3D.step	背下时间步主线
4	_project_contact / _scatter_contact	理解 CFEMP 的区别所在
5	_p2g / _g2p	连接你熟悉的 mpm88.py
6	_update_* / *_internal_force	补齐弱形式与本构
7	__init__ / _hex8_mesh	理解 field shape 从哪里来
8	benchmark_3d.py	理解验证指标如何计算
9	tests/test_plate_impact_3d.py	理解“算对”的证据
10	animate_impact_3d.py	理解状态怎样可视化

14.1 关键代码定位

内容	文件	当前行段
配置与解析解	cfemp/plate_impact_3d.py	26-111
HEX8 网格与 Gauss	同上	114-220
结果数据类	同上	223-273
Taichi field 初始化	同上	276-419
P2G 与网格速度	同上	476-518
接触映射/投影/回写	同上	520-630
应力、内力与 G2P	同上	632-775
step 与 run	同上	862-1033
指标与图片	cfemp/benchmark_3d.py	1-335
动画	cfemp/animate_impact_3d.py	1-297

真正需要默写的只有三块
第一是 step() 的调用顺序；第二是 P2G/G2P 的质量、动量映射；第三是接触点 reduced mass + impulse 的投影。
其他代码大多是几何初始化、存储和验证，可以查着写。

15 做完本教程后，你应该能回答的问题

- 为什么 FEM 节点不是“缓冲器”，而是材料自由度？
- 为什么 MPM 粒子保存历史，而背景网格每步清空？
- 为什么 FEM 表面要映射到 MPM 网格，不能直接拿最近粒子碰撞？
- 为什么接触冲量使用 reduced mass？
- 为什么投影后法向相对速度恰好为 0？
- 为什么 FEM 回写必须做质量归一化？
- 为什么接触在一个 step 内做两次？
- 为什么 P2G 和内力组装必须 atomic_add？
- 为什么测试通过仍不等于真实铝板实验可信？
- 怎样从两个质量点逐步扩展到三维 CFEMP？

15.1 建议的亲手练习

- 练习 A：用 NumPy 写两个质量点的一维投影，随机质量和速度下检查动量守恒。
- 练习 B：单独实现 3D trilinear shape，做 partition of unity 测试。
- 练习 C：把 impact_speed 改为 50、150 m/s，验证弹性应力与速度成正比。
- 练习 D：把 dt 依次减半，画应力与分离时间误差。
- 练习 E：增加摩擦切向投影，但先在斜向单点接触上测试。
- 练习 F：加入 J2 弹塑性与各向同性硬化，复现论文的弹塑性双波。

15.2 下一版合理的工程架构

当你准备加入多物体、曲面接触或塑性时，可以把当前大类拆成四个 data-oriented 组件：FEMBody3D 保存节点/单元，MPMBody3D 保存粒子/网格，ContactCoupler3D 只做界面映射和投影，ExplicitStepper 组织时间积分；Diagnostics 保持纯 Python。

最终判断

你不需要一次性记住所有公式。只要能画出数据流、写出 step()、解释 contact impulse，并知道每个阶段用什么测试卡住错误，就已经具备独立重写和扩展这个 CFEMP 算例的能力。

附录 A 最小伪代码

可从这个 50 行级骨架开始

```
config = Config()
fem = build_hex8_body(config)
mpm = build_particle_body(config)
grid = build_background_grid(config)

for step in range(config.steps):
    grid.clear()
    mpm.p2g(grid)

    vf = map_fem_surface_to_grid(fem, grid)
    project_normal_momentum(vf, grid.mpm_state)
    scatter_impulse_to_fem(fem, grid)

    fem.update_stress()
    mpm.update_stress(grid)
    fem.assemble_internal_force()
    mpm.assemble_grid_force(grid)

    fem.predict_velocity()
    grid.predict_momentum()

    vf_trial = map_fem_surface_to_grid(fem, grid, use_trial=True)
    project_normal_momentum(vf_trial, grid.mpm_state)
    scatter_impulse_to_fem(fem, grid, use_trial=True)

    fem.update_position()
    mpm.g2p(grid)
    diagnostics.record(fem, mpm, grid)
```

附录 B 运行命令

安装、计算、动画和测试

```
python -m pip install -e .
python -m cfemp.benchmark_3d
python -m cfemp.animate_impact_3d
python -m unittest discover -s tests -v
```

附录 C 参考资料

- Y. P. Lian, X. Zhang, Y. Liu. Coupling of finite element method with material point method by local multi-mesh contact method. Computer Methods in Applied Mechanics and Engineering, 200 (2011), 3482-3494.
- A Coupled Finite Element Material Point Method for Large Deformation Problems. 2018 方法章节；本文档所实现的对称板碰撞基准来自其引用的 2011 原始论文。
- 项目仓库：<https://github.com/ecjtusyy/CFEMP>
- 核心实现：cfemp/plate_impact_3d.py；算法说明：docs/method_3d.md；测试：tests/test_plate_impact_3d.py。

范围声明：当前实现是三维线弹性、平直无摩擦界面的验证算例；采用 $2 \times 2 \times 2$ 全积分，不是原作者单点积分加沙漏控制代码的逐行复制。